

Solving rigid body constraints with a small step solver

Guangya Cai
cai00171@umn.edu

1 Introduction

For solving rigid body constraints such as contacts and joints in interactive applications like video games, projected Gauss-Seidel (PGS) method has been the state-of-the-art technique in the industry for a long while and it often works well [7, 8]. However, one notorious problem of this method is its poor rate of convergence for large mass ratios, that is, it does not converge well when rigid bodies involved in the constraint have large differences in their effective masses. There have been some attempts to tackle this problem [11], but performances of these methods are not good enough for interactive applications. Typically, people in the industry get around with this issues with some hacks [27], which make animations less realistic.

Recently, NVIDIA announced their new temporal Gauss-Seidel (TGS) solver and claimed that it converges much better for large mass ratios with low performance losses, compared with the PGS solver [26]. They also had an academic paper [14] published about it. However, the paper itself only demonstrated a position based version of it, which is not the same as the TGS solver. In this work, we utilize the key idea in [14] to implement a small step PGS solver for velocity based rigid body simulation, just as they did in the TGS solver. We verify the convergence claimant for large mass ratios using similar test scenes presented in [26] and confirm that this solver indeed works better for large mass ratios. However, we also find out that new instability issues introduced by this solver, which were briefly discussed in [14], are hard to cope with and downplay the advantage of this solver significantly. We'll discuss these issues in details.

2 Related Work

Rigid body dynamics simulation is an integral and important part of many interactive applications. For some video games (e.g. Angry Birds), it's critical to the gameplay. In the graphic community, early works mostly focus on acceleration based methods [3, 4]. However, the acceleration based method is inherently instable for solving rigid body contacts with the Coulomb friction model, which can be shown by the Painlevé paradox [20] that is well-known in the applied mechanics community. Despite some advantages in the accuracy of the simulation [9], acceleration based methods is not being used in today's interactive applications for its stability issues.

To cope with problems like the Painlevé paradox, starting from 1970s, researchers in the applied mechanics community had turned to work on velocity based methods [22]. The velocity based method was greatly developed and made rigorous by later works [17, 18, 16]. Eventually, some works [25, 2, 19] were made known by people outside of the applied mechanics community. The velocity based method is still the state-of-the-art method for interactive applications.

For the velocity based method, typically, one formulates the problem of solving rigid body constraints into a linear complementarity problem (LCP) with impulses as unknown variables and uses a LCP solver to solve it. Initially, direct solvers such as the Danzig's algorithm [23] and the

Lemke’s algorithm [12] were used. However, the performance of a direct solver is not good enough for interactive applications so people started to turn to look at iterative solvers. At the end, PGS turned out to be the best one and was popularized quickly [7, 8]. Until now, PGS is still the state-of-the-art solver in the industry [21].

Despite the great success of PGS in the video games, it’s also well known that this solver does not converge well when rigid bodies involved in the constraint have large differences in their effective masses, which is the so-called large mass ratio problem. Typically, people in the game industry get around this problem by hacking the mass ratio, such as clamping the mass ratio at the solver level and artificially increasing the moments of inertia to reduce variance of the effective mass in different configurations [27]. Of course, these hacks make animations less realistic and players might notice that in some situations. There also have been some attempts to tackle this problem, such as isolating those hard to solve constraints by applying the Gauss–Seidel splitting (which is also called ”sweeping process” [19] by people in the applied mechanics community), then use a direct solver to solve those sub block linear complementarity problems in each iteration [11]. However, performances of these methods are still not good enough. Recently, the large mass ratio problem has also attracted attentions of people in the applied mechanics community. Some iterative methods with better convergence, such as the interior point method [15] and the alternating direction method of multipliers (ADMM) [28], were made to work for solving rigid body constraints with a different formulazation of the problem. However, the runtime of a single iteration of these methods is still too expensive for real-time applications.

3 Method

In this section, we briefly describe the formulization of the problem and how PGS and small step PGS work. We omit the a lot of details and we won’t discuss how to handle frictions for the simplicity. One can refer to [6] or [1] for more details. One thing worth noting is that for commonly used methods in handling frictions for real-time applications, their convergences lack of theoretical justifications.

3.1 Formulization

Let $q_t = (p_t, Q_t)$ defines the position of a rigid body in generalized coordinates, where p_t is a vector representing the position and Q_t is a quaternion representing the orientation at the time t . Also, let $u_t = (v_t^\top \ w_t^\top)^\top$ defines its velocities (linear and angular) at the time t . It’s easy to see that one can find a linear map H such that $\dot{q}_t = Hu_t$. So, in the following, we’ll omit this linear map and use $\dot{q}_t$ to denote velocities.

For rigid body constraints in this work, in general, they can be classified into two category: $C(q_t^1, q_t^2) = 0$ and $C(q_t^1, q_t^2) \geq 0$. The former one is a bilateral constraint that is often used to represent a joint constraint and the latter one is a unilateral constraint that is often used to represent a contact constraint. In the context of classical mechanics, by the principle of least action, one can incorporate these constraints with a optimization based approached. For bilateral constraints, one can use the method of Lagrange multipliers. And for unilateral constraints, one can use KKT conditions. After applying the the method of Lagrange multipliers or the KKT conditions, we can discrete and then simplify equations by only considering changes of velocities in a single time step, this is also called time-stepping method by the applied mechanics community [24]. For unilateral constraints, we have that

$$J\dot{q}_t + b \geq 0 \perp \lambda \geq 0 \tag{1}$$

where $t' = t + \Delta t$, λ is the vector of contact impulses and we abuse the notation by concatenating all general positions into a single vector. " $\perp$ " denotes the complementary condition that $(J\dot{q}_{t'} + b)^\top \lambda = 0$. Also, J is the jacobian matrix of constraints, that is

$$J = \frac{\partial C}{\partial q} \quad (2)$$

at time t . We abuse the notation by concatenating all general constraints together. Then, by accounting for dynamic equations (Newton–Euler equations), we have the linear complementarity problem of the following form

$$JM^{-1}J^\top \lambda + b' \geq 0 \perp \lambda \geq 0. \quad (3)$$

Where M is a generalized mass matrix containing the rigid body mass properties including the moment of inertia. Now, this problem can be solved by a LCP solver. One thing worth noting is that in most cases, $JM^{-1}J^\top$ is a sparse matrix.

Similar, for bilateral constraints, we have that

$$JM^{-1}J^\top \lambda + b' = 0. \quad (4)$$

By the method of Lagrange multipliers, we don't have further constraints on λ .

The above method computes constraint impulses in order to prevent a violation of constraints due to external forces. However, if the constraints are violated in a different way (e.g. by errors that occur during numerical integration), the method cannot correct this. In this case, one has to use some constraint stabilization methods to correct these errors. Also, in interactive applications, contact constraints are only solved when overlaps between rigid bodies are detected, the stabilization method is needed to resolve penetrations between them. One commonly used stabilization method is the Baumgarte stabilizer [5], which adds an additional term to correct the error.

$$JM^{-1}J^\top \lambda + b' + \gamma \frac{\epsilon}{\Delta t} \geq 0 \perp \lambda \geq 0 \quad (5)$$

where ϵ is the error and γ is a control parameter. This same method can also be used to stabilize bilateral constraints.

Beside the Baumgarte stabilizer, the other popular stabilization method is the post-stabilization method [10]. This method is inherently more stable for contact constraints. However, it has some performance overheads and our experiments shows that it might not be a good choice for the small step solver.

3.2 PGS

The projected Gauss-Seidel method is a variant of the Gauss-Seidel method for the solving linear complementarity problem. The idea is simple, after the i -th entry of unknown variables is solved in a Gauss-Seidel iteration, it adds the following clamping step

$$\lambda_i^{(k+1)} = \text{clamp}(\lambda_i^{(k+1)}, \lambda_i^{lo}, \lambda_i^{hi}) \quad (6)$$

where $\lambda_i^{(k+1)}$ on the right-hand side is the result of Gauss-Seidel iteration for the i -th entry at the $k + 1$ -th iteration. For unilateral constraints, we set $\lambda_i^{lo} = 0$ and $\lambda_i^{hi} = +\infty$ and for bilateral constraints, we set $\lambda_i^{lo} = -\infty$ and $\lambda_i^{hi} = +\infty$. Notice that by setting $\lambda_i^{lo} = -\infty$ and $\lambda_i^{hi} = +\infty$ for all i , the projected Gauss-Seidel method becomes the Gauss-Seidel method for solving linear equations.

In many applications, one usually chooses a convergence criteria and keeps the projected Gauss-Seidel method running until the criteria is met. However, for many real-time applications, a hard limit of the maximum number of iterations is set and the number is usually a small one. So, in many scenarios, the solver does not run enough iterations to converge. One way to make this issue less severe is to warm start the solver using the result of the previous frame. By the convergence behaviour of the PGS [13], with each iteration, the result becomes closer to the solution. If your initial guess is already close, the number of iterations that it would take to converge would be quite small. In some cases, the matrix $JM^{-1}J^T$ wouldn't change dramatically between two adjacent frames so the result of the previous frame could be a good guess.

3.3 Small Step PGS

A common practice in simulation for interactive applications is to use a large time step to do numerical integration and a iterative solver to solve constraints. The idea of a small step solver is quite simple. Instead of using a large time step with n constraint solver iterations, it use n smaller time steps with a single constraint solver iteration for each substep. This simple idea turns to be quite powerful and a small step PGS solver convergence much better at large mass ratio, compared with a PGS solver.

The pseudocode of a small step PGS solver is presented as follow. At the start of each iteration, we compute new $JM^{-1}J^T$ and b' since they are related to the general velocity and the general position of rigid bodies. Next, a single projected Gauss-Seidel iteration is executed. Finally, we compute new general velocity and general position for rigid bodies, which would be processed by the next iteration.

Algorithm 1 Small Step PGS

```

1:  $\Delta t_s \leftarrow \frac{\Delta t}{n}$ 
2: for  $i = 1, 2, \dots, n$  do
3:   Update  $M$  by  $q^i$ .
4:   Update  $J$  by  $q^i$  and  $C$ .
5:   Update  $b$  by  $q^i$  and  $C$ .
6:   Update  $\epsilon$  by  $q^i$  and  $C$ .
7:   Update  $JM^{-1}J^T$  by  $J$  and  $M$ .
8:   Update  $b'$  by  $b, q^i, J, M, F_{ext}$  and  $\Delta t_s$ .
9:    $b^\dagger \leftarrow b' + \gamma \frac{\epsilon}{\Delta t_s}$ 
10:   $\lambda^i \leftarrow \text{PGS-ITERATION}(JM^{-1}J^T, b^\dagger, \lambda^{lo}, \lambda^{hi})$ 
11:   $\dot{q}^{i+1} \leftarrow \text{COMPUTEVELOCITIES}(q^i, \lambda^i, J, M, F_{ext}, \Delta t_s)$ 
12:   $q^{i+1} \leftarrow \text{INTEGRATE}(q^i, \dot{q}^{i+1}, \Delta t_s)$ 
13: end for
```

As can be seen in the algorithm above, we keep C fixed across iterations. This is a critical difference between using a small step solver and simply choosing a small step. By doing this, we don't have to perform collision detection every time step, which itself adds a significant performance overhead. We use a one-shot algorithm to get all contact constraints before the execution of the small step solver. One might argue that some collisions might be missed in doing so but in most cases, missed collisions would be processed in the next frame.

4 Experimental Results

In this section, we showcase several experimental results for large mass ratios and make comparisons between the small step PGS and the PGS solver. The first test case is a chain of rigid bodies, with a heavy one at the bottom of the chain. One side of the chain is fixed and it falls under gravity. The mass ratio is 200 : 1 in our experiment. One can clearly see that the small step solver produces smooth and stable results and the PGS fails to converge even when the maximum number of iterations is set to be 100 times greater.

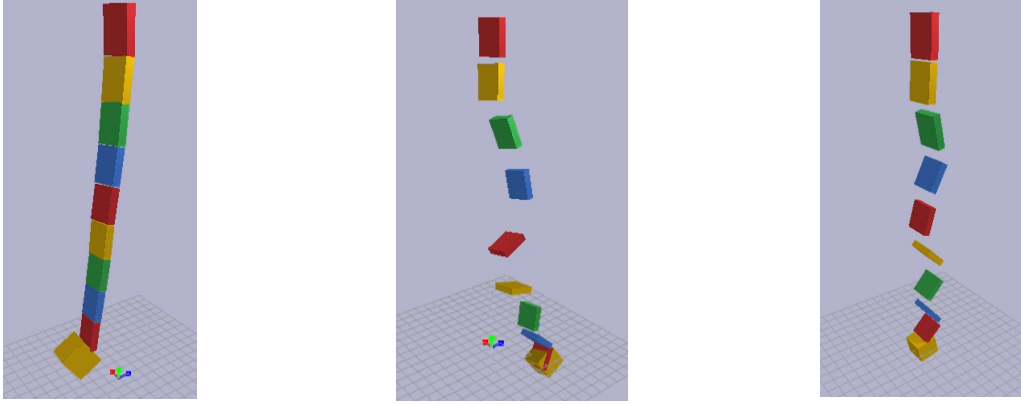


Figure 1: A chain of rigid bodies with a suspended weight creating a mass ratio of 200 : 1. Left: 20 substeps and 1 iteration. Middle: 1 substep and 20 iterations. Right: 1 substep and 2000 iterations.

The second test case is a stack of boxes with a heavy box at the top. The mass ratio is also 200 : 1. For the small step PGS solver, it maintains a stable stack of boxes. As for the PGS solver, the stack collapses quickly.



Figure 2: A stack of boxes with a heavy one at the top creating a mass ratio of 200 : 1. Left: 20 substeps and 1 iteration. Right: 1 substep and 20 iterations.

From these two test cases, one can clearly see that the small step PGS solver convergence better than the PGS solver for large mass ratios.

5 Performance

One strength of the small step solver, compared with other methods for large mass ratios [11, 15, 28], is its performance. The extra runtime needed for the small step solver in each iteration is linear to

the number of constraints, which makes the asymptotic runtime of small step PGS and PGS the same one. We also setup a performance benchmark scene to compare performances. It contains 5 stacks of boxes with 1076 boxes in total. The maximum number of iterations is set to be 10 for both solvers.

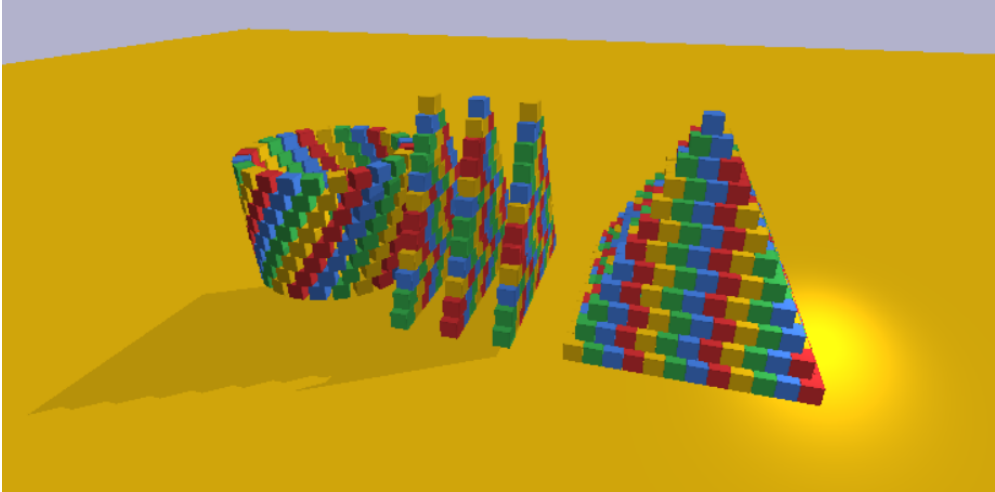


Figure 3: Performance benchmark, 10 iterations for both solvers.

The benchmark result shows that for the small step PGS solver, it takes 23ms to run a single time step, and for the PGS solver, it takes 9ms to run a single time step. If we exclude the time spent for collision detections, the relative performance overheads of the small step solver becomes even higher. One reason causing this is that the PGS solver we use to make comparisons is a well optimized one from the Bullet engine and our small step solver is not that well optimized. We believe the performance overheads can be made lower if more efforts put into it.

6 Limitations and Issues

Even though the small step solver converge better for some large mass ratio cases, that does not mean it works better for all cases. It's known in [14] that this solver introduces new instabilities for its tendency to overaggressive constraint stabilization. This issue is worse for contact constraints, which was also noticed in [14] so they introduced another parameter to control the constraint stabilization. For the Baumgarte stabilizer, one can easily see that it adds energy to the whole system, which makes it inherently unstable. So, one might think the post-stabilization method [10], which stabilizes constraints in the position level and thus is more stable, can be used to deal with these new instabilities issues. Also, it's not hard to implement post-stabilization in a small step manner. However, it turns out that the Baumgarte stabilizer is also critical to get stable results at large mass ratio. It's the extra impulses brought by the Baumgarte stabilizer that makes the stack of boxes not collapsing. Our attempts in getting the post-stabilization method works for the small step solver does not lead to fruitful results. One thing we do in this work to improve the stability is applying position based constraint stabilization in the spirit of the post-stabilization before the execution of the solver (which is also called pre-stabilization by some). By doing this, We hope that some deep penetrations can be removed to begin with so the Baumgarte stabilizer wouldn't add too much energy later on. However, the extra parameter that controls the constraint stabilization is still needed since the Baumgarte stabilizer is still in use.

Also, we notice that if the mass ratio is small, we have to reduce the value of γ to make the simulation stable otherwise the simulation becomes very energetic. Even worse, as noted in [14], the stability of the small step solver is dependent to the number of iterations, which makes it harder to finetune γ . However, this might not be a serious issue for real-time applications since the maximum number of iterations is pretty small.

Finally, the small step solver lacks rigorous convergence analysis that makes it hard to reason with its convergence behaviour. One might notice the similarity between a small step and a small learning rate in the optimization algorithm and this might be a direction to reason with the convergence behaviour of the small step solver. However, it seems like the graphics community does not care too much about the theoretical convergence analysis of a solver.

7 Conclusion

In this work, we implement the small step PGS solver to solve rigid body constraints. Experimental results show that it indeed converges better than the state-of-the-art solver for large mass ratios. However, we also find out that new instabilities introduced by this solver are hard to cope with and surprisingly, these new instabilities are critical elements to get stable results for large mass ratios to begin with. Since these new instabilities are hard to deal with, it leaves end users to decide which solver should be chosen in different scenarios.

References

- [1] Sheldon Andrews, Kenny Erleben, and Zachary Ferguson. Contact and friction simulation for computer graphics. In *ACM SIGGRAPH 2022 Courses*, SIGGRAPH '22, 2022.
- [2] Mihai Anitescu and Florian A Potra. Formulating dynamic multi-rigid-body contact problems with friction as solvable linear complementarity problems. *Nonlinear Dynamics*, 14(3):231–247, 1997.
- [3] David Baraff. Analytical methods for dynamic simulation of non-penetrating rigid bodies. In *Proceedings of the 16th annual conference on Computer graphics and interactive techniques*, SIGGRAPH '89, pages 223–232, 1989.
- [4] David Baraff. Non-penetrating rigid body simulation. In *State of the art reports, Eurographics '93.*, 1993.
- [5] Joachim Baumgarte. Stabilization of constraints and integrals of motion in dynamical systems. *Computer methods in applied mechanics and engineering*, 1(1):1–16, 1972.
- [6] Jan Bender, Kenny Erleben, Jeff Trinkle, and Erwin Coumans. Interactive Simulation of Rigid Body Dynamics in Computer Graphics. In *Eurographics 2012 - State of the Art Reports*. The Eurographics Association, 2012.
- [7] Erin Catto. Iterative dynamics with temporal coherence. Game developer conference, 2005.
- [8] Erin Catto. Fast and simple physics using sequential impulses. Game developer conference, 2006.
- [9] Anindya Chatterjee. On the realism of complementarity conditions in rigid body collisions. *Nonlinear Dynamics*, 20(2):159–168, 1999.

- [10] Michael B Cline and Dinesh K Pai. Post-stabilization for rigid body simulation with contact and constraints. In *2003 IEEE International Conference on Robotics and Automation*, volume 3, pages 3744–3751. IEEE, 2003.
- [11] Erwin Coumans. Exploring MLCP solvers and Featherstone. Game developer conference, 2014.
- [12] Chris Hecker. Lemke’s algorithm, the hammer in your math toolbox? Game developer conference, 2004.
- [13] Alfredo N Iusem. On the convergence of iterative methods for symmetric linear complementarity problems. *Mathematical programming*, 59(1):33–48, 1993.
- [14] Miles Macklin, Kier Storey, Michelle Lu, Pierre Terdiman, Nuttapong Chentanez, Stefan Jeschke, and Matthias Müller. Small steps in physics simulation. In *Proceedings of the 18th Annual ACM SIGGRAPH/Eurographics Symposium on Computer Animation*, SCA ’19, 2019.
- [15] Dario Mangoni, Alessandro Tasora, and Rinaldo Garziera. A primal–dual predictor–corrector interior point method for non-smooth contact dynamics. *Computer Methods in Applied Mechanics and Engineering*, 330:351–367, 2018.
- [16] M.D.P. Monteiro Marques. *Differential inclusions in nonsmooth mechanical problems: Shocks and dry friction*, volume 9. Birkhäuser, 1993.
- [17] Jean Jacques Moreau. Liaisons unilatérales sans frottement et chocs inélastiques. *Comptes rendus hebdomadaires des séances de l’Académie des sciences*, 296:1473–1476, 1983.
- [18] Jean Jacques Moreau. Unilateral contact and dry friction in finite freedom dynamics. In J. J. Moreau and P. D. Panagiotopoulos, editors, *Nonsmooth mechanics and Applications*, pages 1–82. Springer, 1988.
- [19] Jean Jacques Moreau. Numerical aspects of the sweeping process. *Computer methods in applied mechanics and engineering*, 177(3-4):329–349, 1999.
- [20] Paul Painlevé. Sur les lois du frottement de glissement. *Comptes Rendu des Séances de l’Académie des Sciences*, 121:112–115, 1895.
- [21] Jorrit Rouwé. Architecting jolt physics for ‘horizon forbidden west’. Game developer conference, 2022.
- [22] Michelle Schatzman. Sur une classe de problèmes hyperboliques non linéaires. *Comptes Rendus de l’Académie des Sci Série A*, 277:671–674, 1973.
- [23] Russel Smith. Open dynamics engine. <http://www.ode.org/>, 2000.
- [24] David E Stewart. Rigid-body dynamics with friction and impact. *SIAM review*, 42(1):3–39, 2000.
- [25] David E Stewart and Jeffrey C Trinkle. An implicit time-stepping scheme for rigid body dynamics with inelastic collisions and coulomb friction. *International Journal for Numerical Methods in Engineering*, 39(15):2673–2691, 1996.
- [26] Keir Storey, Michelle Lu, and Gordon Yeoman. Physx 4: raising the fidelity and performance of physics simulation in games. Game developer conference, 2019.

- [27] Oliver Strunk. Stop my constraints from blowing up! Game developer conference, 2013.
- [28] Alessandro Tasora, Dario Mangoni, Simone Benatti, and Rinaldo Garziera. Solving variational inequalities and cone complementarity problems in nonsmooth dynamics using the alternating direction method of multipliers. *International Journal for Numerical Methods in Engineering*, 122(16):4093–4113, 2021.